Министерство образования Республики Беларусь
Учреждение образования «Белорусский государственный университет информатики и радиоэлектроники»

Факультет компьютерных систем и сетей
Кафедра информатики
Дисциплина «Модели данных и системы управления базами данных»

ОТЧЕТ
к лабораторной работе №4
на тему:
«REDIS»
БГУИР 6-05-0612-02 67

Выполнил студент группы 353503
КОХАН Артём Игоревич

_____
(дата, подпись студента)

Проверил ассистент каф. информатики
ИОВИЧ Евгения Александровна

_____
(дата, подпись преподавателя)

Минск 2026

1 ИНДИВИДУАЛЬНОЕ ЗАДАНИЕ

Дана схема реляционной базы данных на рисунке 1.1:

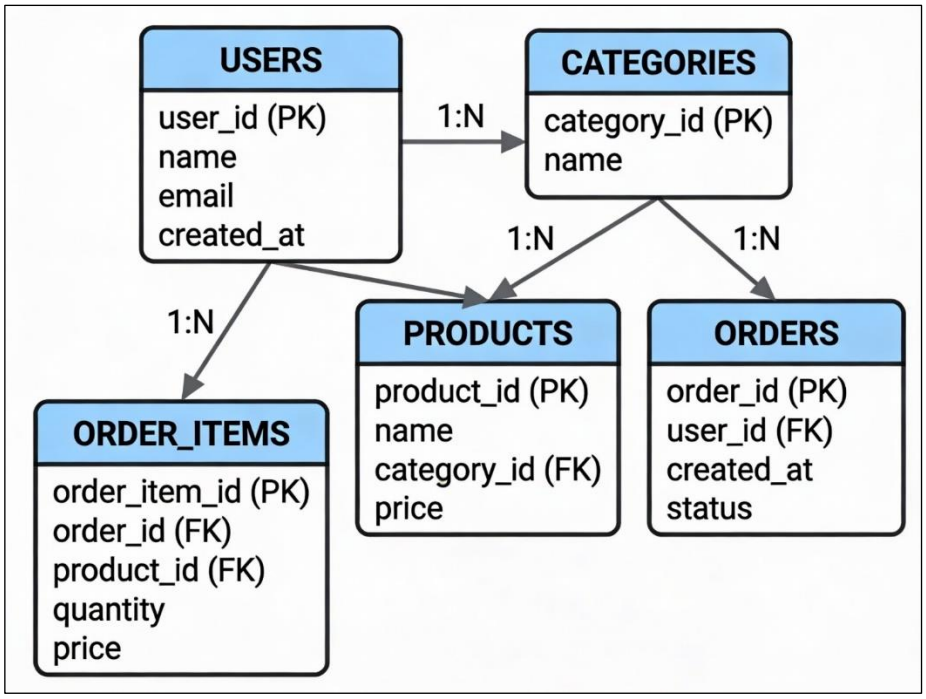

Рисунок 1.1 – Схема реляционной базы данных

Диаграмма показывает:

- **Users** (пользователи) с user_id как первичный ключ
- **Categories** (категории товаров) с category_id как первичный ключ
- **Products** (товары) с внешним ключом category_id на Categories
- **Orders** (заказы) с внешним ключом user_id на Users
- **OrderItems** (позиции заказов) с внешним ключом order_id на Orders и product_id на Products, а также собственным первичным ключом order_item_id для упрощения миграции

Все связи реализованы как один-ко-многим (один пользователь → много заказов, один заказ → много позиций и т.д.).

Прилагаются файлы с данными для импорта.

Необходимо:

- перечислить основные **use cases**:

например, «вывести последние заказы пользователя», «посчитать топ-N популярных товаров», «найти все товары категории», «посчитать выручку по пользователю» и т.д.;

- отметить операции, где в SQL использовались **JOIN, GROUP BY, ORDER BY, LIMIT**.
- Спроектировать ключи и структуру базы данных Redis
  - какие типы данных использовать:
    - Hash для сущности (user:{id}, product:{id});
    - Set для множества (user:{id}:orders, category:{id}:products);

        - Sorted Set для топов (products:by_sales, products:by_rating);
        - List для «ленты» (последние заказы, события).
    - структуру ключей:
        - соглашения вида entity:{id}, entity:{id}:subentity, index:{field}:{value} и т.п.;
- Оформить **таблицу соответствий** «таблица/запрос в SQL → структура данных и ключи в Redis».
- На основе предоставленных данных в CSV написать **скрипт миграции в Redis**:
    - прочитать данные из файлов/БД;
    - создать Hash-объекты для сущностей (пользователи, товары, заказы);
    - заполнить необходимые множества/списки/сортированные множества для индексов и агрегатов.
- Проверить через redis-cli наличие и корректность ключей, (например KEYS user:*, HGETALL user:123, ZRANGE products:by_sales 0 9 WITHSCORES)
- Для указанной предметной области задать **набор бизнес-операций**, которые нужно реализовать с использованием Redis.
    - Получить последние 10 заказов пользователя»
        - хранить идентификаторы заказов в List/Sorted Set user:{id}:orders;
        - реализовать функцию (на выбранном языке) или последовательность команд в redis-cli, возвращающую заказы с деталями.
    - «Топ N товаров по количеству продаж / по выручке»
        - использовать Sorted Set products:by_sales (score = количество проданных единиц) и/или products:by_revenue (score = суммарная выручка);
        - реализовать функции get_top_products_by_sales(n) и get_top_products_by_revenue(n).
    - «Товары по категории + фильтрация по цене»
        - Set category:{id}:products → множество идентификаторов;
        - отдельный Sorted Set products:by_price или хранение цены в Hash + фильтрация приложением.
    - «Похожие пользователи / рекомендации»
        - хранить множество просмотренных/купленных товаров пользователем (user:{id}:viewed, user:{id}:purchased как Sets);
        - использовать операции множеств (SINTER, SUNION, SDIFF) для нахождения пересечений.
- Выбрать одну или две «тяжёлые» операции (например, агрегаты по продажам) и реализовать:
    - ключ кэша, например cache:top_products:by_sales:{N};
    - заполнение кэша (вычисление значения и SET key value EX 60);
    - последующее чтение с проверкой наличия (GET → при null пересчитать и продлить кэш)

2 КРАТКИЕ ТЕОРИТИЧЕСКИЕ СВЕДЕНИЯ

Redis (Remote Dictionary Server) представляет собой резидентную систему управления базами данных класса NoSQL с открытым исходным кодом. В отличие от традиционных реляционных СУБД, Redis хранит данные в оперативной памяти, что обеспечивает высокую производительность операций чтения и записи. Redis поддерживает различные типы данных: строки (strings), списки (lists), множества (sets), упорядоченные множества (sorted sets), хеши (hashes), битовые массивы (bitmaps) и другие.

Реляционные базы данных (PostgreSQL, MySQL, SQLite и другие) используют табличную модель, поддерживают транзакции, сложные JOIN-запросы и обеспечивают целостность данных за счет внешних ключей. Redis, как хранилище "ключ-значение", предлагает гибкую схему, высокую скорость доступа, но ограниченные возможности для сложных запросов и связей между данными. Redis часто используется для кэширования, сессий, очередей и рейтингов.

Типы данных Redis и их применение. Строки (strings) – простейший тип, подходят для хранения отдельных значений. Хеши (hashes) – коллекции пар "поле-значение", идеальны для представления объектов (пользователь, товар). Списки (lists) – упорядоченные коллекции строк, поддерживающие вставку/удаление с обоих концов, используются для лент новостей, очередей. Множества (sets) – неупорядоченные коллекции уникальных строк, подходят для хранения тегов, уникальных элементов. Упорядоченные множества (sorted sets) – множества, где каждый элемент имеет числовой счетчик (score) для сортировки, применяются для рейтингов, топ-листов.

Проектирование структуры данных в Redis требует иного подхода по сравнению с реляционными базами. Вместо нормализации данных используется денормализация и дублирование для повышения скорости чтения. Ключи организуются по соглашению, например, "user:1". Индексы создаются с использованием множеств или упорядоченных множеств, например, "категория:1:товары" для быстрого поиска товаров по категории.

Команды Redis для работы с данными. Основные команды: SET/GET для строк, HSET/HGETALL для хешей, LPUSH/LRANGE для списков, SADD/SMEMBERS для множеств, ZADD/ZRANGE для упорядоченных множеств. Команды для управления ключами: KEYS для поиска по шаблону, DEL для удаления, EXPIRE для установки времени жизни.

Кэширование в Redis – один из основных сценариев использования. Redis позволяет хранить результаты сложных вычислений или частых запросов с временем жизни (TTL), что снижает нагрузку на основную базу данных. Используются команды SETEX для записи с TTL, GET для чтения.

3 ВЫПОЛНЕНИЕ РАБОТЫ

На основе предоставленной схемы базы данных (сущности USERS, CATEGORIES, PRODUCTS, ORDERS, ORDER_ITEMS) были выделены основные сценарии использования (use cases) системы. К ним относятся:

1 Вывод последних заказов пользователя.

2 Расчет топ N популярных товаров по количеству продаж.

3 Расчет топ N популярных товаров по выручке.

4 Поиск всех товаров категории.

5 Фильтрация товаров по категории и цене.

6 Расчет выручки по пользователю.

7 Найти пересечение покупок двух пользователей.

8 Найти объединение покупок двух пользователей.

9 Вывод списка пользователей.

10 Вывод всех категорий товаров.

При реализации этих сценариев в реляционной SQL-базе данных активно используются операции JOIN (для соединения таблиц), ORDER BY (для сортировки) и LIMIT (для ограничения выборки). Например, получение топа товаров требует JOIN таблиц ORDER_ITEMS и PRODUCTS, и сортировки по сумме количества или выручки.

Для обеспечения высокой производительности операций спроектирована структура данных в Redis (смотреть рисунок 3.1), ориентированная на конкретные запросы. Используются Hash-структуры для хранения полных данных сущностей: category:{id}, product:{id}, user:{id}, order:{id}, order_item:{id}.

Для хранения последовательностей идентификаторов используются List-структуры: categories:all_categories содержит список всех category_id, users:all_users содержит список всех user_id, orders:latest_orders содержит список последних order_id.

Для представления множеств идентификаторов применяются Set-структуры: category:{id}:products содержит множество product_id товаров в категории, order:{id}:items содержит множество order_item_id позиций в заказе, user:{id}:purchased содержит множество product_id купленных пользователем товаров.

Для рейтингов и сортировок с числовым score используются Sorted Set-структуры: user:{id}:orders содержит заказы пользователя с score в виде timestamp, products:by_price содержит товары с score равным цене, products:by_sales содержит товары с score равным количеству продаж, products:by_revenue содержит товары с score равным выручке.

Для хранения предварительно вычисленных результатов применяются String-структуры с TTL в качестве кэша: cache:top_products:by_sales:{N} содержит сериализованные данные топа N товаров по продажам с TTL 60 секунд, cache:top_products:by_revenue:{N} содержит сериализованные данные топа N товаров по выручке с TTL 60 секунд.

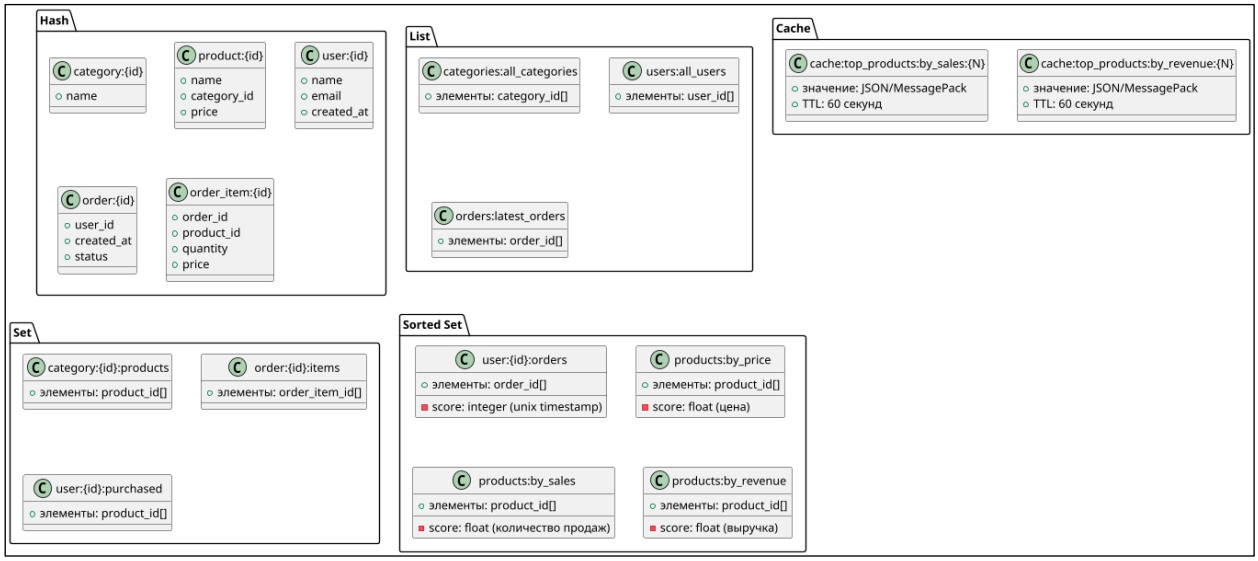

Рисунок 3.1 – Диаграмма структуры Redis

Основная логика отображения SQL-операций на структуры Redis представлена в таблице соответствий (смотреть рисунок 3.2). Диаграмма наглядно показывает, как каждый типовой SQL-запрос заменяется одной или несколькими операциями с определенными ключами Redis. Напомним, что логика отображения будет содержать use cases, которые были определены ранее:

1 Вывод последних заказов пользователя.
2 Расчет топ N популярных товаров по количеству продаж.
3 Расчет топ N популярных товаров по выручке.
4 Поиск всех товаров категории.
5 Фильтрация товаров по категории и цене.
6 Расчет выручки по пользователю.
7 Найти пересечение покупок двух пользователей.
8 Найти объединение покупок двух пользователей.
9 Вывод списка пользователей.
10 Вывод всех категорий товаров.

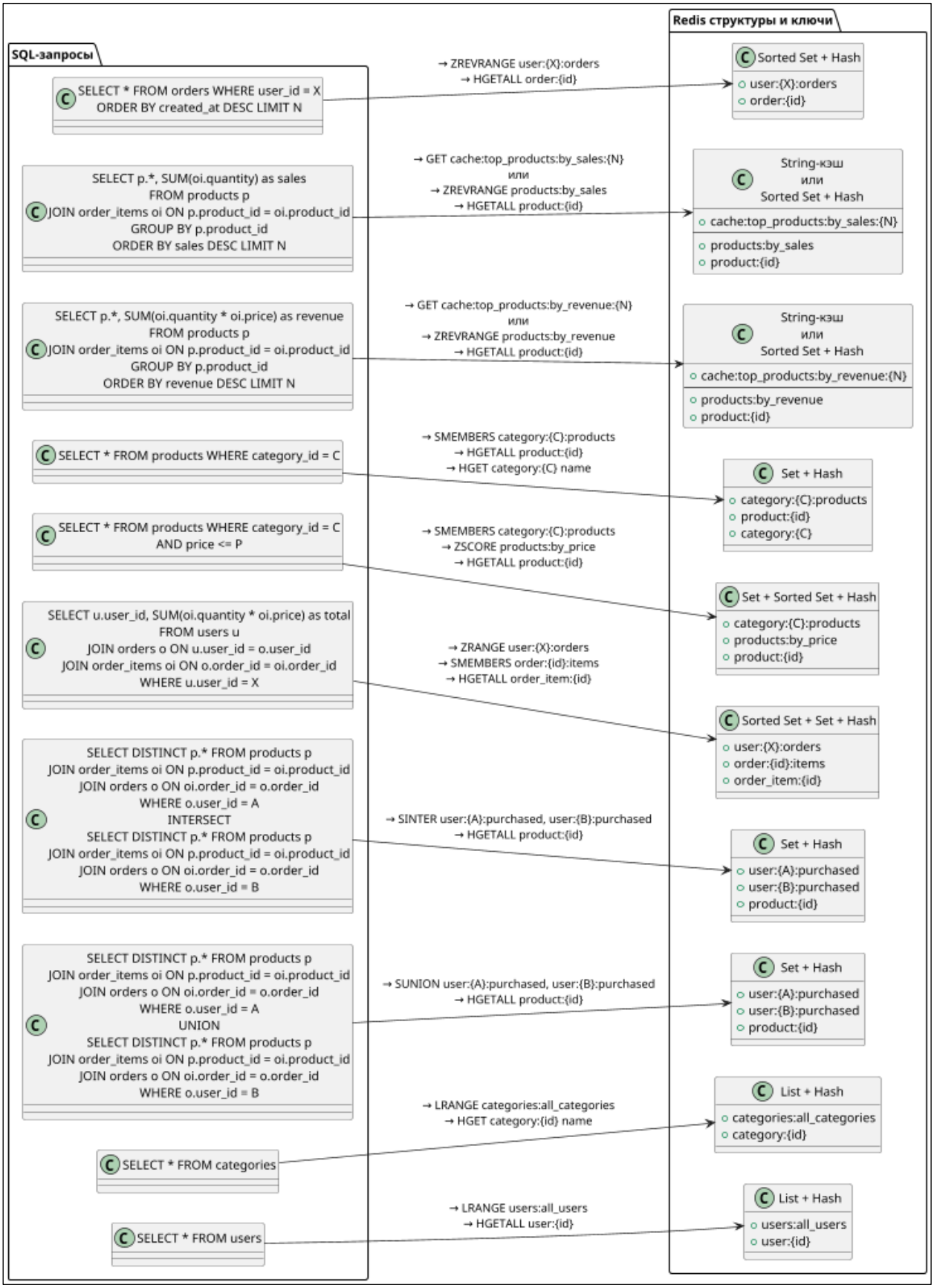

Рисунок 3.2 – Отображение SQL-операций на структуры Redis

Для заполнения Redis данными, к лабораторной работе были прикреплены csv файлы с данными. Миграция данных в redis была реализована на языке python. В результате чего получили, спроектированную ранее, структуру (смотреть рисунок 3.3).

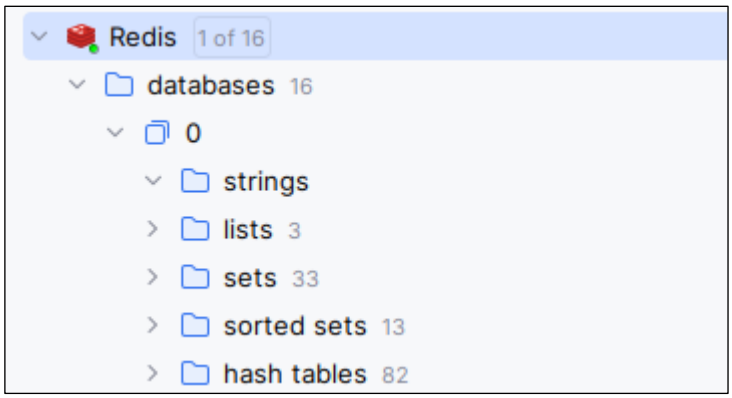

Рисунок 3.3 – Заполненная данными Redis

Целостность базы данных, также, была проверена через redis-cli. Все запланированные структуры данных созданы и содержат ожидаемые данные. На изображении рисунке 3.4 видно выполнение команды KEYS user:*, которая показала наличие всех ожидаемых структур данных для пользователей.

```
127.0.0.1:6379> KEYS user:*
 1) "user:9:orders"
 2) "user:9"
 3) "user:6:orders"
 4) "user:3:orders"
 5) "user:5:orders"
 6) "user:9:purchased"
 7) "user:5:purchased"
 8) "user:4"
```

Рисунок 3.4 – Выполнение команды KEYS user:*

На изображении рисунке 3.5 показан результат выполнения команды для одной из Sorted Set структур ZRANGE products:by_price 0 4 WITHSCORES, которая демонстрирует корректную работу для индексации товаров по цене.

```
127.0.0.1:6379> ZRANGE products:by_price 0 4 WITHSCORES
 1) "5"
 2) "19.99"
 3) "12"
 4) "25.99"
 5) "19"
 6) "29.99"
 7) "15"
 8) "34.99"
 9) "11"
10) "38.99"
```

Рисунок 3.5 – Выполнение команды для одной из Sorted Set структур

Для реализации бизнес-операций с использованием Redis было создано приложение на Python с интерфейсом командной строки, часть кода можно увидеть в листинге 3.1.

Листинг 3.1 – Реализация интерфейса командой строки

```
def main():
    try:
        r.ping()
        print("✓ Подключение к Redis успешно")
    except:
        print("✗ Ошибка подключения к Redis")
        return

    while True:
        print_menu()

        try:
            choice = input("\nВыберите операцию (0-9) or q to exit: ").strip()

            if choice == 'q':
                break

            elif choice == '0':
                user_id = input("Введите ID пользователя: ").strip()
                limit = input("Сколько заказов показать: ").strip()
                limit = int(limit) if limit else 10
                get_last_user_orders(user_id, limit)

            elif choice == '1':
                N = input("Введите N для топа: ").strip()
                N = int(N) if N else 10
                get_top_products_by_sales_cached(N)

            elif choice == '2':
                N = input("Введите N для топа: ").strip()
                N = int(N) if N else 10
                get_top_products_by_revenue_cached(N)

            elif choice == '3':
                category_id = input("Введите ID категории: ").strip()
                get_products_by_category(category_id)

            elif choice == '4':
```

```
            category_id = input("Введите ID категории: ").strip()
            max_price = float(input("Введите максимальную цену: ").strip())
            filter_products_by_category_and_price(category_id, max_price)

        elif choice == '5':
            user_id = input("Введите ID пользователя: ").strip()
            get_user_revenue(user_id)

        elif choice == '6':
            user1_id = input("Введите ID первого пользователя: ").strip()
            user2_id = input("Введите ID второго пользователя: ").strip()
            get_intersection_purchases(user1_id, user2_id)

        elif choice == '7':
            user1_id = input("Введите ID первого пользователя: ").strip()
            user2_id = input("Введите ID второго пользователя: ").strip()
            get_union_purchases(user1_id, user2_id)

        elif choice == '8':
            get_all_users()

        elif choice == '9':
            get_all_categories()

        else:
            print("Неверный выбор. Попробуйте снова.", end="\n\n")

        print(end='\n')

    except ValueError as e:
        print(f"Ошибка ввода: {e}", end="\n\n")
    except Exception as e:
        print(f"Ошибка: {e}", end="\n\n")
```

Вывод меню выбор осуществляется перед каждым предложением выбрать одну из предложенных функций (смотреть рисунок 3.6).

```
0.  Вывести последние заказы пользователя
1.  Посчитать топ N популярных товаров по количеству продаж
2.  Посчитать топ N товаров по выручке
3.  Найти все товары определённой категории
4.  Отфильтровать товары по категории и цене
5.  Посчитать выручку по пользователю
6.  Найти пересечение покупок двух пользователей
7.  Найти объединение покупок двух пользователей
8.  Вывод списка пользователей
9.  Вывод всех категорий товаров
q = Выход

Выберите операцию (0-9) or q to exit:
```

Рисунок 3.6 – Меню выбора бизнес-операций

Операция "Получить последние 10 заказов пользователя" реализована с использованием Sorted Set, где ключ user:{id}:orders хранит идентификаторы заказов с score в виде временной метки. Функция get_last_user_orders использует команду ZREVRANGE для получения отсортированных по

времени заказов и дополняет их деталями из Hash-структур order:{id}. Результат работы функции можно увидеть на рисунке 3.7.

```
Выберите операцию (0-9) or q to exit: 0
Введите ID пользователя: 1
Сколько заказов показать: 10

Последние 2 заказов пользователя 1:
1. Заказ #2 - shipped - 2024-03-20 16:45:00
2. Заказ #1 - completed - 2024-03-15 14:30:00
```

Рисунок 3.7 – Результат работы операции "Получить последние 10 заказов пользователя"

Для операций "Топ N товаров по количеству продаж" и "Топ N товаров по выручке" используются Sorted Set-структуры products:by_sales и products:by_revenue, где score представляет соответственно количество проданных единиц и суммарную выручку. Реализованы функции get_top_products_by_sales_cached и get_top_products_by_revenue_cached. Результат одной из функций можно на увидеть ниже, на рисунке 3.8.

```
Выберите операцию (0-9) or q to exit: 1
Введите N для топа: 5
Кэш пуст, вычисляем данные...
Данные сохранены в кэш на 60 секунд

Топ-5 товаров по продажам:
1. Джинсы классические - 3.0 шт. - 49.99 руб.
2. Футболка хлопковая - 3.0 шт. - 19.99 руб.
3. Набор для йоги - 3.0 шт. - 34.99 руб.
4. Платье вечернее - 2.0 шт. - 79.99 руб.
5. Sony WH-1000XM5 - 2.0 шт. - 349.99 руб.
```

Рисунок 3.8 – Результат работы операции "Топ N товаров по количеству продаж"

Операция "Товары по категории + фильтрация по цене" использует Set category:{id}:products для хранения идентификаторов товаров категории и Sorted Set products:by_price для фильтрации по цене. Функция filter_products_by_category_and_price выполняет фильтрацию на стороне приложения (смотреть рисунок 3.9).

```
Выберите операцию (0-9) or q to exit: 4
Введите ID категории: 1
Введите максимальную цену: 500

Товары категории 'Электроника' до 500.0 руб.:
1. Sony WH-1000XM5 - 349.99 руб.
```

Рисунок 3.9 – Результат работы операции "Товары по категории + фильтрация по цене"

Для операции "Похожие пользователи" используются Set-структуры user:{id}:purchased, хранящие купленные пользователем товары. Реализованы функции get_intersection_purchases и get_union_purchases, которые используют операции множеств Redis SINTER и SUNION для нахождения пересечения и объединения покупок пользователей. Одна из функций представлена ниже, на рисунке 3.10.

```
Выберите операцию (0-9) or q to exit: 6
Введите ID первого пользователя: 1
Введите ID второго пользователя: 5
Пересечение покупок user1_id=1 и user2_id=5:
ID: 3, MacBook Air M2 - 1199.99
```

Рисунок 3.10 – Результат работы операции "Похожие пользователи"

Для обеспечения производительности тяжелых операций "Топ N товаров по количеству продаж" и "Топ N товаров по выручке" реализовано кэширование. Используются ключи кэша cache:top_products:by_sales:{N} и cache:top_products:by_revenue:{N} с TTL 60 секунд. Универсальная функция get_cached_data проверяет наличие данных в кэше, при отсутствии вычисляет их и сохраняет с указанным временем жизни. На рисунке 3.11, показан пример, когда данные взяты из кэша.

```
Выберите операцию (0-9) or q to exit: 1
Введите N для топа: 3
Данные получены из кэша (ключ: cache:top_products:by_sales:3)

Топ-3 товаров по продажам:
1. Джинсы классические - 3.0 шт. - 49.99 руб.
2. Футболка хлопковая - 3.0 шт. - 19.99 руб.
3. Набор для йоги - 3.0 шт. - 34.99 руб.
```

Рисунок 3.11 – Результат работы операции "Топ N товаров по количеству продаж с кэшированием"

ВЫВОД

В ходе выполнения лабораторной работы были успешно изучены и применены на практике принципы проектирования и реализации структуры данных в Redis. Была спроектирована и реализована структура данных, использующая Hash-структуры для хранения полных данных сущностей: пользователей, товаров, категорий, заказов и позиций заказов. Для хранения последовательностей идентификаторов применены List-структуры, такие как список всех пользователей и список всех категорий. Для представления множеств идентификаторов использованы Set-структуры, включая множества товаров в категориях и купленных товаров пользователями. Для рейтингов и сортировок созданы Sorted Set-структуры с числовым score, такие как рейтинг товаров по продажам, выручке и цене.

Разработано Python-приложение с интерфейсом командной строки, реализующее все запланированные бизнес-операции. Разработана универсальная функция get_cached_data, которая проверяет наличие данных в кэше, при отсутствии вычисляет их и сохраняет с указанным временем жизни. Реализовано кэширование для тяжелых операций, таких как топ товаров по продажам и выручке, с использованием ключей cache:top_products:by_sales:{N} и cache:top_products:by_revenue:{N} и TTL 60 секунд.

Полученные знания и навыки позволяют проектировать и реализовывать структуры данных в Redis, оптимизированные под конкретные бизнес-операции, использовать различные типы данных Redis для решения различных задач и реализовывать механизмы кэширования для повышения производительности.